

Budget Allocation in a Formula 1 Strategy Competition

Kiran Patel, 201698246

Supervisor: Dr Cédric Beaume
March 27, 2026

Contents

1	Introduction	2
2	Model construction and validation	3
2.1	From the competition brief to a simplified model	3
2.2	Driver signing model	3
2.3	Performance and reliability model	4
2.4	Monte Carlo simulation and objectives	5
3	Results	6
3.1	Validation of the draft simulation	6
3.2	Reliability investment and DNF probability	6
3.3	Coarse-grid comparison of mean points and win probability	7
3.4	Local refinement of the win-probability objective	9
3.5	Season outcome distributions	9
4	Robustness and sensitivity	10
4.1	Robustness of the mean-points strategy	10
4.2	Robustness of the win-probability strategy	11
4.3	Performance under different opponent styles	12
5	Interpretation and limitations	14
6	Conclusion	16
	Bibliography	17
A	Core simulation code	18
B	Objective evaluation and search code	23
C	Robustness and opponent-sensitivity code	28
	Declarations of Academic Integrity & GenAI use	33

1 Introduction

Budget allocation is central to success in Cédric Beaume’s Formula 1 Strategy Competition. In the financial stage of the competition, each team is given a fixed budget of 8ι , where ι is an imaginary currency unit used only within the competition framework. That budget must be divided between four departments: marketing (to improve driver-signing chances), chassis development (to enhance car structure), engine development (to boost power), and reliability (to reduce mechanical failures) (Beaume, 2025). The core question studied in this report is therefore: how should a team allocate this budget across the four departments to maximise its season performance?

The competition itself extends beyond the financial stage into a race-strategy stage involving pit-stop planning. However, this report focuses deliberately on the opening allocation decision and its season-level consequences. The aim is not to reproduce every feature of the full competition, but to isolate the mathematical structure of the budget-allocation problem and investigate it carefully. In particular, the report studies how the initial budget split shapes driver recruitment, car performance, reliability, and ultimately championship outcomes across a ten-race season.

This problem is inherently stochastic. Driver recruitment depends on probabilistic contest outcomes driven by marketing budgets, while season results depend on uncertain failures and the realised driver line-up. Monte Carlo simulation is, therefore, a natural modelling tool. More broadly, simulation-based approaches are well established in motorsport and decision analysis under uncertainty. Bekker and Lotz (2009) use discrete-event simulation to study Formula One race strategy, while Heilmeier et al. (2020) apply Monte Carlo methods to circuit-racing simulation with probabilistic effects. More generally, Tolk (2022) emphasises that simulation-based optimisation is especially useful when the solution space is shaped by uncertainty and complex interactions. Recent motorsport work continues in the same direction: Agud and Thraves (2024) study pit-stop strategy using dynamic programming and game theory, and Avella et al. (2026) develop a recent Formula 1 simulation framework centred on race-strategy optimisation. The present project is much simpler than those models, but it uses the same broad methodological idea: repeated stochastic simulation to evaluate strategic choices.

A second important theme is that there is no single notion of an optimal allocation unless the objective is specified first. In this report, two main objectives are considered: maximising expected season points and maximising the probability of winning the championship. These objectives need not agree. Indeed, Agud and Thraves (2024) show in a different Formula 1 context that strategies become more risk-seeking when the objective is framed in terms of winning odds rather than a smoother average-performance criterion. One of the central findings of the present project is that the same phenomenon appears here: the best mean-points strategy is not the same as the best win-probability strategy.

Although this is a stylised competition model rather than a model of real Formula 1 finance, the underlying question is still connected to wider discussions about spending structure and competitive performance. Empirical work on actual Formula 1 budgets suggests that spending patterns matter materially for team outcomes (Kallimogiannis and Massouros, 2023; Loník and Kotrba, 2024). At the same time, real Formula 1 operates under a detailed cost-cap regime and a regulatory framework that is far more complex than the simple 8ι game considered here (FIA, 2024, 2025). The abstraction used in this report is therefore intentional: it sacrifices realism in order to make the core allocation trade-offs transparent and analysable.

The report is organised as follows. Section 2 translates the competition rules into a simplified stochastic model, explaining the driver-signing law, the performance and reliability model, the simulation procedure, and the optimisation objectives. Section 3 presents the main results: validation of the recruitment simulation, analysis of the reliability law, comparison of mean points and win probability on a coarse grid, local refinement of the win-probability strategy, and comparison of the season-outcome distributions. Section 4 examines robustness to one-at-a-time perturbations

and to different opponent styles. Section 5 interprets the findings, explains the main modelling choices, and discusses limitations and future work. Section 6 concludes.

2 Model construction and validation

2.1 From the competition brief to a simplified model

The competition brief specifies three stages: a financial stage, a race-strategy stage, and a ten-race championship (Beaume, 2025). This project isolates the first of these stages and asks how the initial budget allocation affects season outcomes. The resulting simplified model links budget allocation to three mechanisms:

1. driver-signing probabilities through marketing,
2. race performance through chassis and engine spending,
3. mechanical failure probability through reliability spending.

These mechanisms then determine stochastic season outcomes through repeated simulation.

This simplification was a deliberate modelling choice. A more detailed race-level model, including explicit lap times and pit stops, would have mirrored the full competition more closely, but would also have introduced substantial extra structure unrelated to the main question of how to allocate the initial budget. By treating the financial stage as the main decision variable and replacing the rest of the competition with a season-level simulator, the model preserves the central strategic trade-offs while remaining tractable.

The budget variables satisfy the competition constraint:

$$m + c + e + r = 8,$$

where m , c , e , and r are the spending levels on marketing, chassis, engine, and reliability, respectively. Each variable must be non-negative. The purpose of the model is therefore to map each feasible allocation (m, c, e, r) to one or more season-performance measures.

2.2 Driver signing model

The first stochastic component is driver recruitment. The competition brief states that if team i invests m_i in marketing, then its probability of signing the driver currently available in the draft is

$$p_i = \frac{m_i^h}{\sum_{j=1}^n m_j^h}, \quad (2.1)$$

where n is the number of teams and $h = 3.2$ is a competition-intensity parameter (Beaume, 2025). A larger value of h makes the contest more concentrated: small differences in marketing budget then produce larger differences in signing probability.

The draft is sequential. Stronger drivers are signed first, then the next strongest, and so on until each team fills its two seats. Once a team has signed both of its drivers, it drops out of the contest for the remaining drivers. This sequential structure matters because a strong marketing position increases the chance of securing the highest-ability drivers early.

An important early step in the project was to test the implementation of this signing mechanism in a smaller two-team setting. In that special case, the analytic signing probability is easy to compute directly from equation (2.1), so the Monte Carlo recruitment code can be checked against a known theoretical benchmark. This was useful both for debugging and for building confidence in the simulator before extending the model to the full eight-team season.

In the final season model, marketing is therefore interpreted primarily as a probabilistic gateway to stronger drivers. The report does not attempt to add further marketing effects beyond recruitment. That choice keeps the role of marketing transparent and in line with the competition rules. The simulator uses the published driver roster for the current competition season, with each driver assigned a fixed ability score on the competition scale from 0 to 5. These ability values enter directly into the recruitment and performance stages of the model, so the optimisation results should be interpreted as conditional not only on the race calendar and competition rules, but also on this particular driver market.

2.3 Performance and reliability model

The competition brief states that chassis and engine spending directly translate into corresponding car strengths, while reliability spending changes the probability of a mechanical failure (Beaume, 2025). The brief also defines a race-performance measure that combines driver ability, chassis strength and engine strength:

$$\text{Performance} = \frac{3(c_d \text{ Driving} + c_c \text{ Chassis} + c_e \text{ Engine})}{c_d + c_c + c_e}, \quad (2.2)$$

where c_d , c_c , and c_e are circuit-specific coefficients for the importance of driving, chassis and engine, respectively. For example, a balanced circuit uses $c_d = c_c = c_e = 1$, while more specialised circuits place extra weight on a particular component.

In the simulator, each driver's race score is computed from equation (2.2) using the driver's signed ability together with the team's chassis and engine budgets. This preserves the key competitive feature of the brief: different circuit types reward different car designs. Engine-heavy or chassis-heavy allocations can therefore be advantageous in some races but not in others. In principle, chassis and engine enter equation (2.2) in a broadly symmetric way: if a circuit weights them equally, then one additional unit of chassis or engine spending contributes equally to the performance score. However, that symmetry is broken by the specific race calendar used in the competition, as the ten circuits do not all weigh the two components equally. The final simulator therefore uses the published competition-season inputs – including the current value of h , the current driver roster, the number of teams, and the ten-race circuit list – rather than an abstract generic season. As a result, the later optimisation results should be interpreted as being conditional on this particular season setup.

Reliability is modelled using the competition's mechanical-failure law

$$p_{\text{DNF}}(r) = \frac{[1 - \text{erf}(0.7r - 1.8)]^2}{5}, \quad (2.3)$$

where $p_{\text{DNF}}(r)$ is the probability that a driver records a Did Not Finish due to mechanical failure, r is the reliability budget, and erf is the Gaussian error function (Beaume, 2025). The error function is a smooth S-shaped function, so here it produces a sharp drop in failure risk over a middle range of r , followed by diminishing returns once reliability is already fairly high.

In each simulated race, drivers are ordered by performance score and then subject to independent DNF events according to equation (2.3). Drivers who finish are awarded points according to the competition scoring system. Because the model uses a 16-driver field, points are awarded to the top eight finishers on the 10, 8, 6, 5, 4, 3, 2, 1 scale, corresponding to the 13–16 driver case in the brief (Beaume, 2025). This is not the standard modern Formula 1 World Championship scoring system; it is the competition's own scoring rule for this field size. Team season points are then defined as the sum of the points scored by that team's two drivers across the ten-race championship.

2.4 Monte Carlo simulation and objectives

For any feasible allocation (m, c, e, r) , the model simulates many full seasons. Each simulated season includes:

1. sequential driver recruitment using the marketing contest rule,
2. conversion of the resulting driver line-up into race-level performance values,
3. ten races with circuit-dependent performance and DNF events,
4. aggregation of Team A's season points and whether Team A wins the constructors' championship.

Monte Carlo repetition then yields empirical estimates of season-level quantities of interest.

Two main objective functions are considered. The first is expected season points:

$$J_{\text{mean}}(m, c, e, r) = \mathbb{E}[\text{Team A season points}]. \quad (2.4)$$

This favours allocations that perform well on average and typically rewards consistency.

The second objective is championship win probability:

$$J_{\text{win}}(m, c, e, r) = \mathbb{P}(\text{Team A wins the constructors' championship}). \quad (2.5)$$

This is a more risk-sensitive objective. An allocation can have a slightly lower average score but still be attractive under J_{win} if it creates more high-upside seasons.

A coarse grid search was first used to explore the feasible budget space broadly. Candidate allocations from promising regions were then examined more carefully. In principle, both objectives could have been refined locally. In practice, only J_{win} was refined in detail. This was a defensible choice for three reasons. First, the coarse-grid mean-points optimum already lay at a competition-feasible one-decimal allocation. Second, the mean-points landscape appeared comparatively broad and stable, so the mean-points recommendation did not seem to depend on a sharply isolated local peak. Third, the championship-win objective is binary and therefore inherently noisier under Monte Carlo simulation, so local refinement added more value there than it would have for J_{mean} . This does not prove that no nearby mean-points allocation could do slightly better, but it does justify focusing the limited refinement effort where it was most informative.

Throughout the report, error bars represent Monte Carlo uncertainty from a finite number of simulated seasons rather than uncertainty in the competition rules themselves. For mean season points, the reported 95% confidence intervals are based on the standard normal approximation:

$$\bar{x} \pm 1.96 \frac{s}{\sqrt{N}},$$

where $\bar{x}$ is the sample mean across N simulated seasons and s is the sample standard deviation. For championship win probability, the corresponding intervals are based on the binomial standard error

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{N}}.$$

For some local and perturbation comparisons, paired simulations with common random numbers were also used in the code so that two nearby allocations were compared under the same random seeds, reducing comparison noise. These uncertainty estimates are not the main focus of the report, but they are important for interpreting whether small visible differences are likely to be meaningful.

A final methodological point is that verification was treated as part of the modelling process rather than as an afterthought. The code was checked using symmetric sanity checks, direct

comparison of simulated signing probabilities against the theoretical law, repeated inspection of the reliability curve, and controlled one-at-a-time experiments in which a single budget component was perturbed while the others were held as fixed as possible. This incremental checking process was crucial in distinguishing genuine trade-offs from coding errors and in supporting the credibility of the later optimisation results.

3 Results

3.1 Validation of the draft simulation

Before using the season simulator for optimisation, the recruitment component was checked against the theoretical signing law. In the two-team case, equation (2.1) gives an explicit analytic probability that Team A signs the stronger driver as a function of the two marketing budgets and the competition-intensity parameter h . If the Monte Carlo recruitment code is correct, its empirical estimate $\hat{p}_A$ should agree closely with the theoretical value p_A .

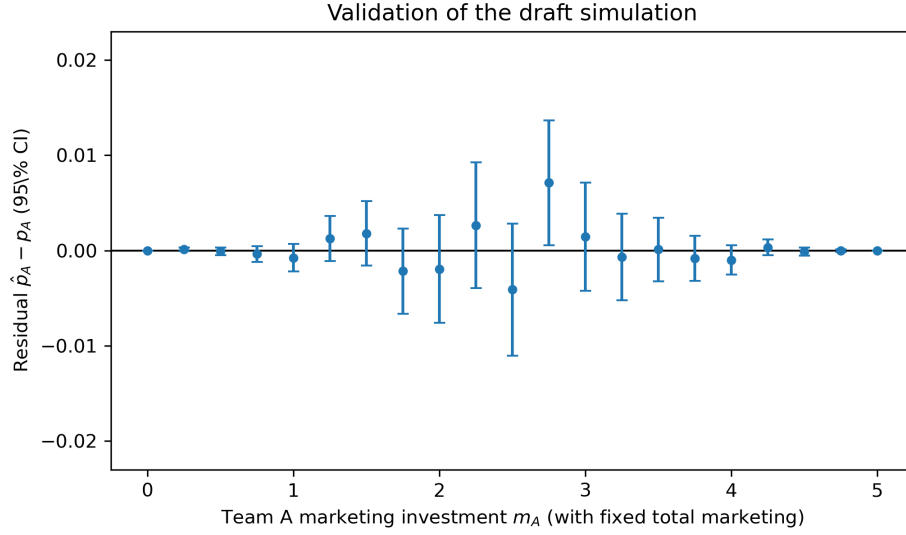


Figure 1: Validation of the draft simulation. The figure shows the residual $\hat{p}_A - p_A$ between the Monte Carlo estimate $\hat{p}_A$ and the theoretical signing probability p_A , with 95% confidence intervals, as Team A's marketing investment varies while total marketing is kept fixed.

Figure 1 shows that these residuals remain close to zero across the full range. The confidence intervals are narrow overall and widest near the middle of the sweep, where p_A is closest to 0.5. This is exactly what should happen for a binomial estimate, since the variance $p(1-p)/N$ is largest near $p = 0.5$ and smallest near $p = 0$ or $p = 1$. The figure, therefore, provides a strong verification that the recruitment stage is functioning as intended.

This check matters because marketing is a major driver of later results. If the signing law had been implemented incorrectly, downstream conclusions about optimal budget allocation would have been unreliable. Figure 1 supports using the recruitment simulation as the stochastic starting point for the season model.

3.2 Reliability investment and DNF probability

The second central model component is the relation between reliability spending and mechanical-failure risk. Equation (2.3) defines a nonlinear map from the reliability budget r to the probability of a DNF event.

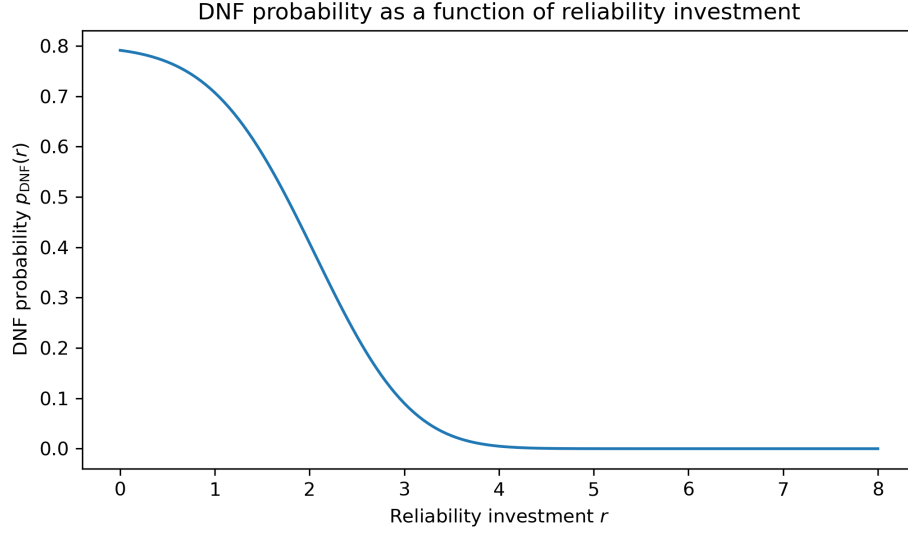


Figure 2: DNF probability as a function of reliability investment. The curve follows the competition law in equation (2.3).

Figure 2 shows two important features. First, failure risk is very high when r is small, so underinvesting in reliability is costly. For example, increasing r from 0 to 1 only reduces p_{DNF} from about 0.79 to 0.71, so a token reliability investment still leaves the team highly exposed. Second, the curve has strong diminishing returns. Moving from $r = 2$ to $r = 3$ reduces p_{DNF} much more sharply, from about 0.41 to about 0.09. Beyond $r = 4$, the DNF probability is already close to zero, so further spending has very little marginal effect.

This shape helps explain several later results. Reliability is essential up to a moderate level because very low values create an unsustainably large retirement risk. However, once that risk has been reduced sufficiently, further reliability spending becomes increasingly inefficient compared with spending the same amount on pace or recruitment. The optimisation results repeatedly reflect this trade-off.

3.3 Coarse-grid comparison of mean points and win probability

The feasible allocation space was first explored using a coarse grid. For each candidate budget split (m, c, e, r) , the simulator estimated both Team A's expected season points and Team A's championship win probability.

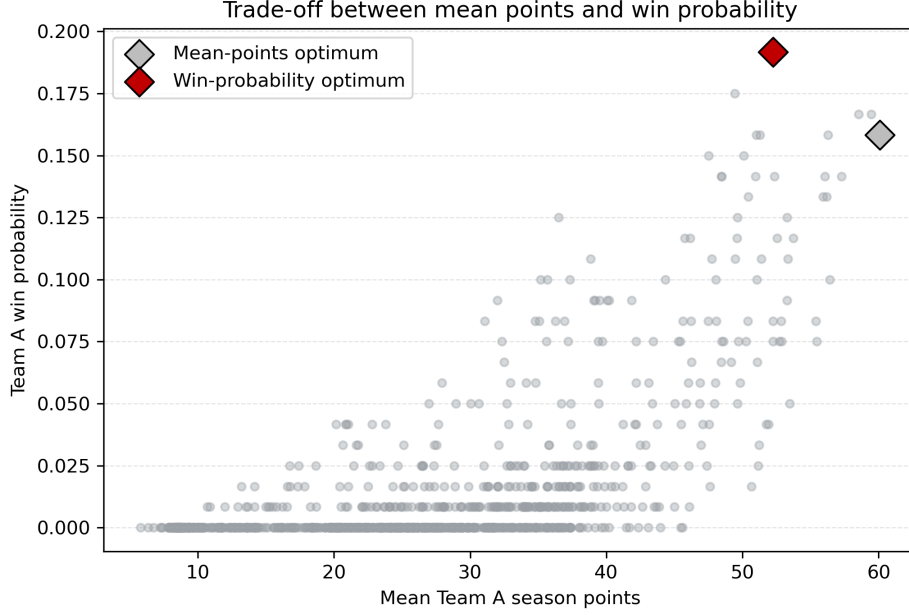


Figure 3: Trade-off between mean season points and win probability across the coarse-grid search. Each point represents one feasible allocation. The grey diamond marks the mean-points optimum (2.5, 0, 3.0, 2.5), and the red diamond marks the chosen win-probability allocation (2.1, 0, 3.4, 2.5).

Figure 3 reveals two main features. First, the point cloud has a clear upper-right frontier. In other words, the strongest allocations lie along a boundary of trade-offs rather than being spread uniformly across the plane. Calling this frontier “Pareto-like” is appropriate here: once an allocation is near that boundary, improving one objective typically requires giving up some performance on the other. A second useful observation is that many sampled points sit much closer to the horizontal axis than to the upper edge of the plot. This reflects the fact that a moderate season-points total is much easier to achieve than a substantial championship-win probability; most allocations can score some points, but comparatively few win the title often.

Second, the highlighted grey and red optima do not coincide. The best mean-points allocation is

$$(m, c, e, r) = (2.5, 0, 3.0, 2.5),$$

whereas the chosen win-focused allocation is

$$(m, c, e, r) = (2.1, 0, 3.4, 2.5).$$

This separation is one of the project’s central findings. Optimising J_{mean} favours a steadier balance, while optimising J_{win} shifts the allocation toward a more aggressive engine-heavy configuration with lower marketing.

A further feature of Figure 3 is that both highlighted allocations place zero budget in the chassis and instead concentrate the remaining performance spending in the engine. This should not be read as meaning that the chassis is intrinsically unimportant. Rather, in the performance law, chassis and engine play comparable roles when circuits weight them equally, but the symmetry between them is broken by the particular circuit mix used in this season. For the published ten-race schedule and the associated competition parameters, the engine side of that trade-off proved more favourable in the optimisation search, so the strongest sampled allocations pushed their discretionary car-development budget toward engine rather than chassis.

The figure also helps explain why only J_{win} was locally refined. The coarse-grid mean-points optimum already sits on a visibly strong region of the frontier, and later robustness analysis shows that nearby allocations remain competitive. By contrast, the red, win-focused region is more delicate and therefore worth examining at a finer resolution.

3.4 Local refinement of the win-probability objective

The coarse search identified a promising region for J_{win} , concentrated near $c = 0$ and $r = 2.5$. Within that region, the main remaining trade-off was between marketing m and engine e . A local refinement sweep was therefore performed by varying m , fixing $c = 0$ and $r = 2.5$, and adjusting e so that the total budget remained 8.

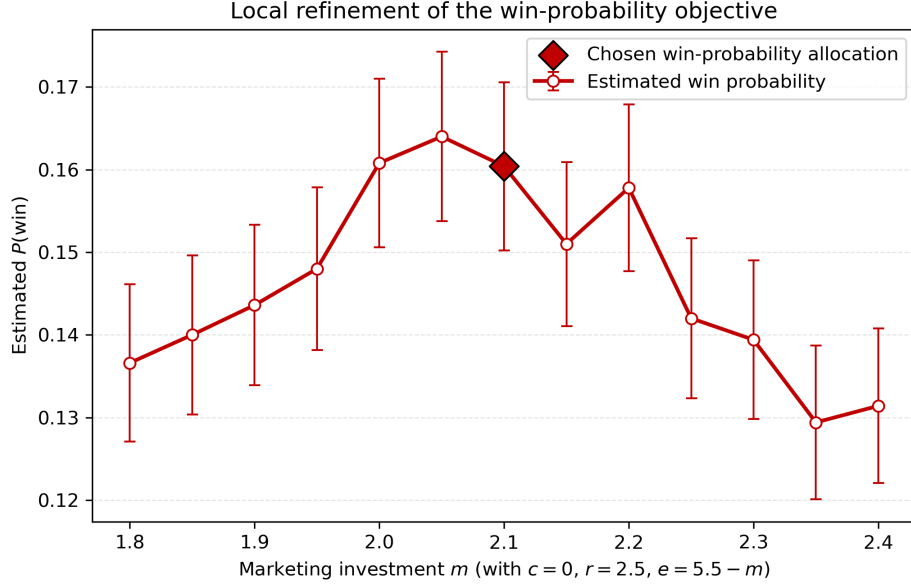


Figure 4: Local refinement of the win-probability objective. The red curve shows the estimated championship win probability as marketing investment varies, with the engine adjusted to keep the total budget fixed. The red diamond marks the selected competition-feasible win-probability allocation $(2.1, 0, 3.4, 2.5)$. Error bars are 95% Monte Carlo confidence intervals.

Figure 4 shows that the strongest-performing region is concentrated in a fairly narrow band of marketing values around $m \approx 2.0$ to 2.1 . The highest sampled point in the refined sweep occurs at

$$(m, c, e, r) = (2.05, 0, 3.45, 2.5).$$

However, that is not the final recommendation, for two reasons. First, the competition only allows one-decimal budget allocations, so 2.05 is not feasible in the actual game. Second, the neighbouring points at $m = 2.0$, $m = 2.05$, and $m = 2.1$ have overlapping confidence intervals, so the apparent peak is not sharp enough to justify treating 2.05 as meaningfully superior in practice.

For that reason, the chosen competition-feasible win-probability allocation is

$$(m, c, e, r) = (2.1, 0, 3.4, 2.5).$$

Figure 4 is still valuable because it shows that the chosen red point lies within a small near-optimal neighbourhood rather than at an isolated spike. This is strategically reassuring: the recommendation does not depend on hitting an unrealistically precise value.

3.5 Season outcome distributions

Average values alone do not tell the full story of how two strategies differ. To understand the practical consequences of the two recommended allocations, it is useful to compare their entire distributions of season outcomes.

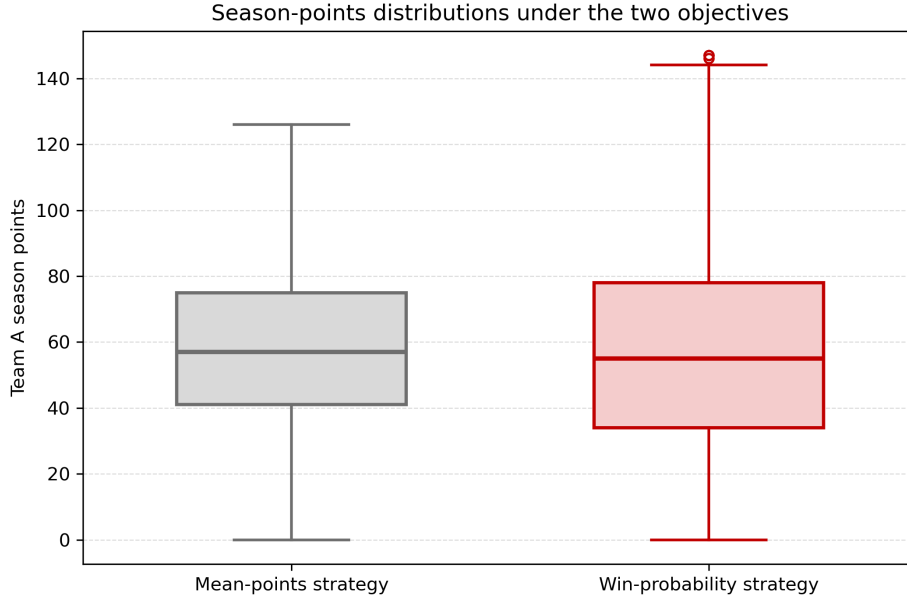


Figure 5: Distribution of Team A season points under the two recommended strategies. The grey boxplot corresponds to the mean-points strategy and the red boxplot to the win-probability strategy.

Figure 5 shows a clear difference in risk profile. The mean-points strategy has the higher centre and the tighter overall spread: its simulated mean is 58, its median is 57, its 10th percentile is 26, and its 90th percentile is 90. By contrast, the win-probability strategy has a slightly lower centre but substantially wider dispersion: its simulated mean is 57, its median is 55, its 10th percentile is 16, and its 90th percentile is 98.

These summary values sharpen the visual interpretation of the boxplots. The grey strategy gives a higher typical score and a noticeably better lower tail, since its 10th percentile is ten points higher. The red strategy, however, gives more upside, since its 90th percentile is higher and its upper tail extends much further. In short, the red strategy accepts more bad seasons in exchange for more very strong seasons.

This is exactly why the two objectives disagree. The mean-points allocation is better at scoring reliably across many seasons, while the win-probability allocation is more volatile but produces more extreme high-end outcomes. The difference is therefore not just numerical but strategic: the grey and red allocations represent two different competitive philosophies.

4 Robustness and sensitivity

4.1 Robustness of the mean-points strategy

A useful recommendation should not depend on an implausibly exact choice of parameters. To test this, the mean-points allocation (2.5, 0, 3.0, 2.5) was subjected to small one-at-a-time percentage perturbations in marketing and reliability, with the total budget kept fixed at 8. In the code, the compensating adjustment is absorbed by chassis and engine: when marketing or reliability is reduced, the freed budget is split equally between chassis and engine; when marketing or reliability is increased, the required budget is first taken equally from chassis and engine, and if that would make one of them negative, it is taken from engine alone.

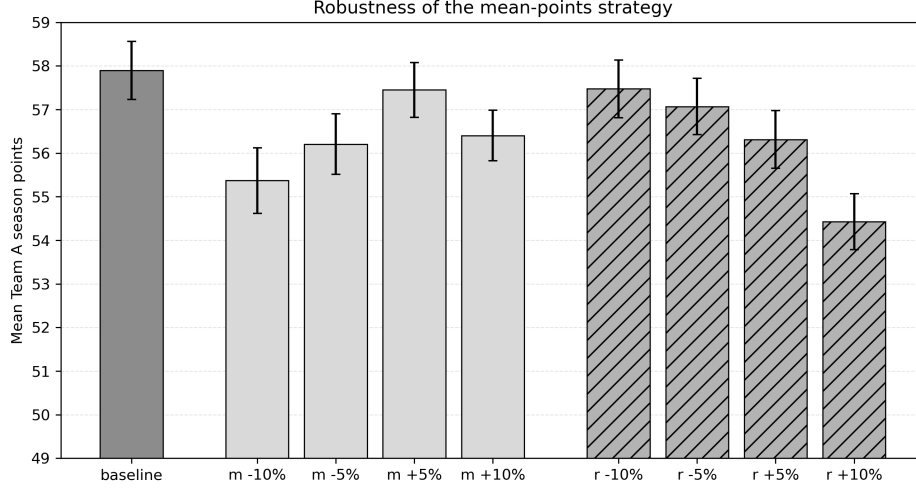


Figure 6: Robustness of the mean-points strategy to one-at-a-time budget perturbations. The baseline grey bar is the mean-points allocation (2.5, 0, 3.0, 2.5); the remaining bars show small perturbations in marketing or reliability. Error bars are 95% Monte Carlo confidence intervals for mean season points.

Figure 6 shows that the mean-points strategy is reasonably stable. Most nearby perturbations remain competitive, which suggests that the grey optimum is part of a broader plateau of strong performance rather than a fragile, isolated spike. This is one of the main reasons why a separate local refinement of J_{mean} was not essential: the surrounding region already looks broad on the coarse search and remains broad here under perturbation.

Not all changes are equally harmless, however. Relative to the baseline of about 58 points, the most damaging perturbations are a 10% reduction in marketing and a 10% increase in reliability. These lower the mean by about 2.5 points and 3.5 points respectively. The latter behaviour is especially consistent with the diminishing-returns reliability law in Figure 2: once reliability is already moderate, extra spending there costs pace or recruitment without buying much further protection against DNFs. Because the baseline has $c = 0$, positive perturbations in marketing or reliability are effectively funded almost entirely by reducing engine. The robustness results should therefore be interpreted as testing trade-offs against engine-heavy performance, rather than against a fully flexible redistribution across all departments.

Overall, Figure 6 supports the grey recommendation. The allocation is not uniquely optimal in a strict mathematical sense, but it is part of a robust neighbourhood of strong average performance.

4.2 Robustness of the win-probability strategy

The same one-at-a-time perturbation analysis was then applied to the chosen red win-probability allocation (2.1, 0, 3.4, 2.5). The same redistribution rule is used here as in Section 4.1. Since the chosen win-probability allocation also has $c = 0$, positive perturbations in marketing or reliability are again funded mainly by reductions in engine. This is important for interpretation, because a bar such as $m+10\%$ is not an isolated increase in marketing alone: it is a joint move toward more marketing and less engine.

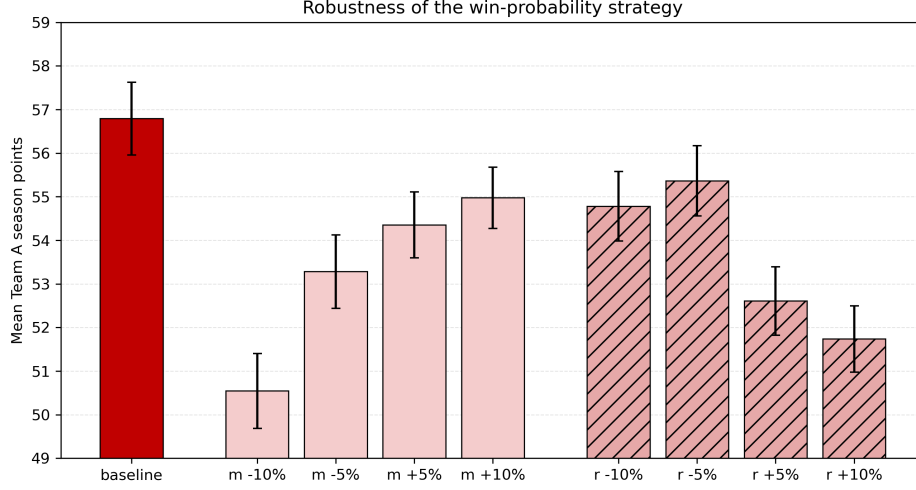


Figure 7: Robustness of the win-probability strategy to one-at-a-time budget perturbations. The baseline red bar is the chosen win-probability allocation (2.1, 0, 3.4, 2.5); the remaining bars show small perturbations in marketing or reliability. Error bars are 95% Monte Carlo confidence intervals for mean season points.

A subtle but important point is that Figure 7 is plotted in mean season points, not in win probability. That is fair as a supporting comparison, because it shows how much average performance is sacrificed when the red strategy is nudged away from its chosen balance. However, it is also slightly incomplete: since the red strategy was not selected to maximise mean points, this figure should not be read as a full robustness analysis of J_{win} itself. A fuller treatment would also show the effect of the perturbations on championship win probability or on paired differences in J_{win} . Under the report's page and figure constraints, Figure 7 is best interpreted as a diagnostic of average-performance sensitivity rather than a complete red-objective analysis.

With that interpretation in mind, the pattern is still informative. On this mean-points scale, the chosen red baseline is the strongest of the nearby allocations shown in Figure 7. Relative to the baseline of about 57 points, a 10% reduction in marketing lowers the mean by just over 6 points, whereas a 10% increase in marketing still remains nearly 2 points below baseline. The asymmetry is therefore real, but it should not be described as a gain from extra marketing. Instead, it shows that the chosen allocation is locally well-balanced: reducing marketing is especially harmful, while increasing it at the expense of engine does not improve the average score enough to beat the baseline.

The reliability perturbations tell a similar story. Modest reductions in reliability are less damaging than increases, but all of the reliability changes shown still sit below the baseline on this mean-points metric, with the largest losses occurring for $r+5\%$ and $r+10\%$. Figure 7 should therefore be interpreted as local evidence that the selected red allocation is internally coherent, even though it was originally chosen to maximise J_{win} rather than J_{mean} .

Taken together, Figures 5 and 7 tell a consistent story. The win-probability strategy is more uneven: it offers a higher ceiling but is also more exposed to adverse deviations from its intended balance.

4.3 Performance under different opponent styles

So far, the optimisation results have assumed balanced opponents with allocation (2.0, 2.0, 2.0, 2.0). To test how sensitive the recommendations are to rival behaviour, the two selected Team A strategies were compared against five stylised opponent profiles:

- balanced: (2.0, 2.0, 2.0, 2.0),
- marketing-heavy: (3.2, 1.6, 1.6, 1.6),

- chassis-heavy: (1.6, 3.2, 1.6, 1.6),
- engine-heavy: (1.6, 1.6, 3.2, 1.6),
- reliability-heavy: (1.6, 1.6, 1.6, 3.2).

In each “heavy” case, one department receives 3.2, which is double the even-split level of 1.6, and the remaining three departments share the rest equally. These are stylised stress tests rather than claims about what all opponents would actually do.

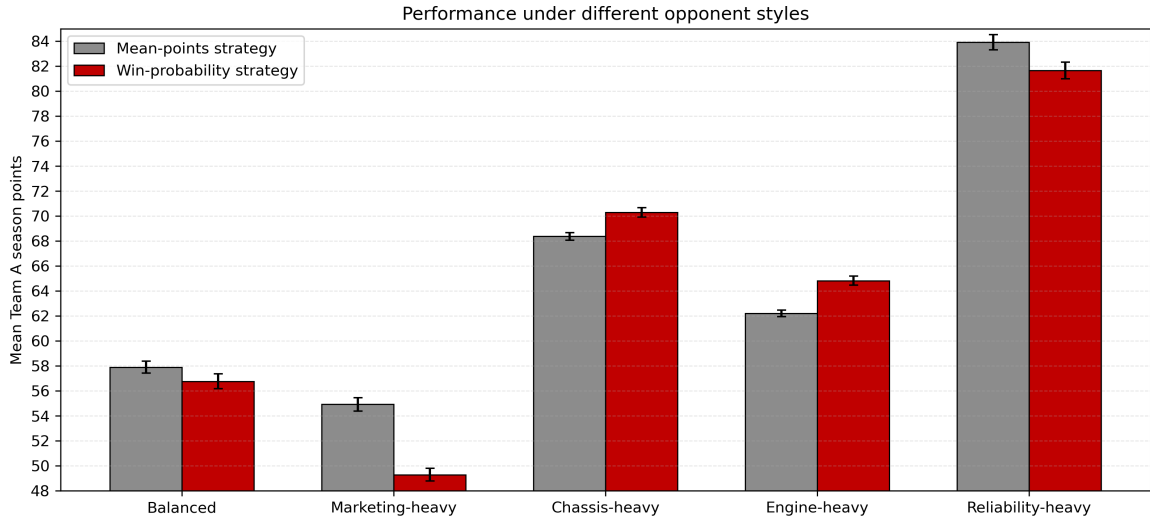


Figure 8: Performance of the two recommended strategies under different opponent styles. Grey bars show the mean-points strategy, and red bars show the win-probability strategy. Opponent styles are the balanced profile (2.0, 2.0, 2.0, 2.0) and four heavy-investment cases in which one department receives 3.2, and the other three each receive 1.6. Error bars are 95% Monte Carlo confidence intervals for mean season points.

Figure 8 shows that the environment matters. Three patterns stand out. First, the overall scoring level varies a great deal across scenarios. Team A scores worst against marketing-heavy opponents, which is intuitive because those opponents make the driver-signing contest harder. By contrast, both Team A strategies score much more highly against reliability-heavy and chassis-heavy opponents, where rival resources are being pulled away from other performance-relevant departments.

Second, the ranking between the grey and red strategies is not uniform across all these stylised environments when performance is measured by mean season points. The grey strategy is slightly better against balanced opponents and clearly better against marketing-heavy opponents. However, in the five-style stress test, the red strategy is very competitive, and appears to edge ahead on mean points, against the chassis-heavy and engine-heavy opponent styles. That is a useful game-theoretic insight: if an opponent field were known in advance to be skewed toward those styles, the aggressive red strategy would be easier to justify even beyond its original J_{win} objective.

Third, against reliability-heavy opponents, both Team A strategies perform very strongly overall, but the steadier grey strategy still appears preferable on average. So if one already knew the opponent allocation profile, the risky red strategy would be most defensible against balanced, chassis-heavy, or engine-heavy fields, but much less attractive against marketing-heavy and reliability-heavy fields.

These scenarios should not be over-interpreted. Real competitor fields would almost certainly be mixed rather than identical. Nevertheless, the exercise is useful because it shows that the effect size – and sometimes even the preferred strategy under a secondary metric like mean points – can

depend strongly on opponent style. The main recommendations are therefore conditional on the competitive environment, not universal truths.

This also highlights an important modelling point. The main optimisation results earlier in the report were obtained against a balanced opponent baseline, so the reported grey and red recommendations are conditional on that reference environment. Had the optimisation instead been carried out against a systematically marketing-heavy, chassis-heavy, engine-heavy, or reliability-heavy field, different optimal allocations could have emerged. In practice, however, the true opponent field is unknown in advance. Using a balanced baseline is therefore a defensible neutral choice: if competitors adopt a mixture of different styles, their aggregate behaviour may plausibly lie closer to balanced than to any one extreme profile.

5 Interpretation and limitations

The main result of the project is that optimal budget allocation depends materially on the objective function. Under the mean-points criterion, the selected allocation is

$$(m, c, e, r) = (2.5, 0, 3.0, 2.5),$$

which yields about 58 expected season points and a championship win probability of about 37%. Under the win-probability criterion, the selected competition-feasible allocation is

$$(m, c, e, r) = (2.1, 0, 3.4, 2.5),$$

which yields a lower mean of about 57 points but a higher championship win probability of about 41%. The gap between these solutions is not a small numerical accident: it reflects a genuine structural difference between rewarding consistency and rewarding upside. It is also worth stressing that both recommended allocations are conditional on the specific season encoded in the model. The simulator uses the competition's current ten-race schedule, the current value of the signing parameter h , the current team count, and the published driver market for that season. In that setting, the engine–chassis trade-off consistently tilted toward engine, which is why both selected allocations ended up with $c = 0$. A different race calendar, a different driver pool, or a different competitive environment could shift that balance and make a positive chassis allocation more attractive.

If one overall recommendation had to be made for the competition, the grey mean-points strategy is the stronger default choice. It has the highest average performance, the highest median season score, the strongest lower tail, and the most stable robustness profile. That recommendation is strengthened by the fact that the surrounding mean-points region is broad rather than knife-edge. This is also why a separate local refinement of J_{mean} was not essential. The coarse-grid mean-points optimum was already competition-feasible, it sat on a visibly strong part of the frontier, and later checks showed that nearby perturbations remain competitive. By contrast, the red win-probability recommendation was more sensitive and therefore more worth refining locally.

At the same time, the red strategy remains entirely defensible when the explicit goal is to maximise the chance of winning the title rather than to score most strongly on average. Figure 5 shows why: the red allocation has a lower centre but a fatter upper tail. It is built to trade away some steadiness in exchange for more exceptionally strong seasons. That is not a weakness in itself; it is simply a different objective-specific choice.

The report also intentionally focuses on a defensible subset of outputs rather than on every possible simulation diagnostic. The final selection of figures – recruitment validation, the DNF curve, the global trade-off cloud, one local win-probability refinement, two robustness scans, and an opponent-style stress test – was chosen to support the central claims as directly as possible within a finite report length. Many additional outputs could have been shown, but not all of them

were equally useful for the main argument. In that sense, the report is selective by design rather than incomplete by accident.

Several limitations should nevertheless be acknowledged. First, the model studies only the financial allocation stage and not the full race-strategy stage of the competition. Second, it simplifies race performance to a brief-based scoring law rather than simulating lap-by-lap racing or explicit pit-stop decisions. Third, it assumes known driver abilities, circuit types and failure law parameters. Fourth, although the search methodology is systematic, it is still based on a coarse-grid screen plus targeted refinement rather than a formal global continuous optimisation algorithm. Fifth, some supporting comparisons, especially Figure 7, deliberately inspect mean points rather than the original optimisation objective, so they should be read as complementary diagnostics rather than complete objective-specific analyses. A related limitation is that the principal optimisation stage assumes balanced opponents. That assumption is reasonable as a neutral baseline when the field is unknown, but it is still an assumption. The opponent-style analysis later in the report shows that changing the surrounding field can change the quantitative performance of the recommended strategies, and in some stylised cases can even change which strategy looks more attractive under a secondary metric such as mean points. This suggests that, had the optimisation been carried out under a different opponent baseline, the resulting recommended allocation might also have been different.

There are also important limits to how far the model should be compared with actual Formula 1. Real teams operate within a detailed cost cap and governance structure overseen by the Fédération Internationale de l'Automobile's (FIA) Cost Cap Administration (FIA, 2025). The Financial Regulations define a complex set of covered costs, exclusions, reporting requirements and compliance mechanisms (FIA, 2024). Historical and empirical work on Formula 1 budgets suggests that spending structure, budget caps, and research-and-development emphasis all matter in ways that are richer than the simple competition game examined here (Kallimogiannis and Massouros, 2023; Loník and Kotrba, 2024). The present model, therefore, should not be interpreted as a realistic representation of modern Formula 1 finance. It is a stylised strategic model built for a competition setting.

Future work could extend the project in several directions. The most obvious next step would be to include the race-strategy stage explicitly, so that financial allocation and pit-stop decisions are modelled jointly rather than separately. That would move the project closer to recent race-strategy work in the literature (Aguad and Thraves, 2024; Avella et al., 2026). A second extension would be to study alternative objectives beyond J_{mean} and J_{win} , such as quantile-based objectives, downside-risk penalties, or multi-objective criteria that reward both consistency and title-winning upside. A third extension would be to replace the stylised opponent tests by mixed populations of opponent styles or adaptive opponents. A fourth option would be to explicitly include parameter uncertainty, for example, by varying driver abilities, circuit schedules, or the reliability law itself. A fifth would be to compare the current coarse-grid-plus-refinement approach against continuous or metaheuristic optimisation methods. Finally, one could revisit the full named-driver draft more explicitly, include weather- or safety-car-style race shocks, or compare across different historical driver pools and seasons. Any of these would increase realism and scope, but also substantially increase model complexity.

Despite these limitations, the project shows that a relatively simple stochastic model can still generate non-trivial and practically useful insights. In particular, it clearly highlights four ideas: objective choice matters, reliability has strong diminishing returns, consistency and upside are distinct strategic aims, and recommendations can depend on the surrounding competitive environment. Those are meaningful conclusions even in a stylised setting.

6 Conclusion

This report studied a simplified Formula 1 budget-allocation problem based on Cédric Beaume's strategy competition. A fixed budget was allocated across marketing, chassis, engine and reliability, and a season-level Monte Carlo simulator was used to evaluate the resulting performance. The model was validated against the theoretical signing law and then used to study reliability, objective trade-offs, local win-focused refinement, robustness, and sensitivity to opponent styles.

The central conclusion is that the optimal allocation depends on what "optimal" means. If the aim is to maximise expected season points and maintain relatively stable performance, the mean-points allocation

$$(2.5, 0, 3.0, 2.5)$$

is the strongest overall recommendation. If instead the explicit goal is to maximise the probability of winning the championship, the competition-feasible win-probability allocation

$$(2.1, 0, 3.4, 2.5)$$

is the appropriate objective-specific choice, even though it is less steady on average.

More broadly, the project shows that carefully designed simplified models can still support useful strategic thinking. The main value of the model is not just that it outputs two candidate allocations, but that it clarifies the trade-offs behind them: recruitment versus pace, reliability versus diminishing returns, and consistency versus upside. In a competition setting characterised by uncertainty, that clarification is itself a valuable result.

Bibliography

- Avella, P., Borruso, M., Marino, M. and Masone, A. (2026). A simulation framework for Formula 1 race strategy based on pit-stop optimization. *Optimization Online* (preprint). Available at: <https://optimization-online.org/2026/02/a-simulation-framework-for-formula-1-race-strategy-based-on-pit-stop-optimization/> (Accessed: 26 March 2026).
- Aguad, F. and Thraves, C. (2024). Optimizing pit stop strategies in Formula 1 with dynamic programming and game theory. *European Journal of Operational Research*, 319(3), pp. 908–919.
- Beaume, C. (2025). *Formula 1 Strategy Competition*. Competition brief, University of Leeds. Available at: <https://cbeaume.com/download/F1StratCompetition.pdf> (Accessed: 21 March 2026).
- Bekker, J. and Lotz, W. (2009). Planning Formula One race strategies using discrete-event simulation. *Journal of the Operational Research Society*, 60(7), pp. 952–961.
- FIA (2024). *Formula 1 Financial Regulations*, Issue 22, 11 December 2024. Available at: https://www.fia.com/sites/default/files/fia_formula_1_financial_regulations_-_issue_22_-_2024-12-11.pdf (Accessed: 26 March 2026).
- FIA (2025). Cost Cap Administration. Available at: <https://api.fia.com/events/fia-formula-one-world-championship/season-2025/cost-cap-administration> (Accessed: 26 March 2026).
- Heilmeier, A., Graf, M., Betz, J. and Lienkamp, M. (2020). Application of Monte Carlo methods to consider probabilistic effects in a race simulation for circuit motorsport. *Applied Sciences*, 10(12), 4229.
- Kallimogiannis, V. and Massouros, C. (2023). Predictions on the impact of the budget cap on Formula 1 teams' performance. *Ratio Mathematica*, 47, pp. 445–468.
- Loník, I. and Kotrba, V. (2024). The key importance of spending on research and development in Formula One. *Managerial and Decision Economics*, 45(1), pp. 70–79.
- Tolk, A. (2022). Simulation-based optimization: implications of complex adaptive systems and deep uncertainty. *Information*, 13(10), 469.

A Core simulation code

This appendix contains selected excerpts of the Python code used to implement the final season-level model described in Sections 2 and 3. The purpose is to document the key model components in a compact and readable way. The excerpt focuses on the recruitment/signing mechanism, the performance law, the reliability law, and the main race and season simulation loops. Auxiliary plotting code, file input/output, and other non-essential infrastructure are omitted for clarity.

The following excerpt shows the main model components: recruitment/signing, the reliability function, the performance calculation, and the core race and season simulation structure.

```
1 import math
2 import numpy as np
3
4 # =====
5 # Appendix A: core season-level model
6 # Distilled from the final working scripts.
7 # =====
8
9 h = 3.2
10 TOTAL_BUDGET = 8.0
11 SEATS_PER_TEAM = 2
12 POINTS_16 = [10, 8, 6, 5, 4, 3, 2, 1] + [0] * 8
13
14 OPP_BASE = {"marketing": 2.0, "chassis": 2.0, "engine": 2.0, "reliability": 2.0}
15
16 drivers = [
17     ("Ricardo Raul (BRA)", 5.0),
18     ("Fabrizio Panduri (ITA)", 4.4),
19     ("Luigi Disanti (ITA)", 4.2),
20     ("Philipp Krafft (GER)", 3.8),
21     ("Joris Gootjes (NED)", 3.7),
22     ("Alfredo Alcarria (SPA)", 3.6),
23     ("Akiko Tomiko (JAP)", 3.5),
24     ("Allan Johnson (GBR)", 3.4),
25     ("Jiang Lin (CHI)", 2.8),
26     ("Germain Grisel (FRA)", 2.4),
27     ("Fabian Metzger (SWI)", 1.8),
28     ("Simao Faria (POR)", 1.5),
29     ("Alfonso Llave (SPA)", 1.2),
30     ("Michel Fabron (FRA)", 0.9),
31     ("Armand Brice (FRA)", 0.3),
32     ("Miko Lautela (FIN)", 0.0),
33 ]
34
35 circuits = [
36     {"name": "Spain", "type": "balanced"},
37     {"name": "Belgium", "type": "chassis_engine"},
38     {"name": "USA", "type": "engine"},
39     {"name": "Japan", "type": "driver_chassis"},
40     {"name": "Canada", "type": "balanced"},
41     {"name": "Hungary", "type": "chassis"},
42     {"name": "Monaco", "type": "driver"},
43     {"name": "Italy", "type": "engine"},
44     {"name": "France", "type": "balanced"},
45     {"name": "Australia", "type": "balanced"},
46 ]
```

```

47
48
49 # =====
50 # Draft-signing model
51 # =====
52
53 def two_team_signing_probability(m_A, m_B, h_param=h):
54     """
55     Analytic two-team signing probability used in the validation stage.
56     """
57     A_term = max(0.0, float(m_A)) ** h_param
58     B_term = max(0.0, float(m_B)) ** h_param
59     total = A_term + B_term
60     if total == 0.0:
61         return 0.5
62     return A_term / total
63
64
65 def contest_probs(marketing_dict, eligible_teams, h_param=h):
66     """
67     Compute contest probabilities among eligible teams:
68      $p_i = m_i^h / \sum_j m_j^h$ 
69     """
70     m = np.array([float(marketing_dict[t]) for t in eligible_teams], dtype=float)
71     m = np.clip(m, 0.0, None)
72
73     weights = m ** h_param
74     total = weights.sum()
75
76     if total == 0.0:
77         return np.ones(len(eligible_teams)) / len(eligible_teams)
78
79     return weights / total
80
81
82 def sign_drivers_sequential(marketing_dict, drivers_list, seats_per_team, h_param,
83                             rng):
84     """
85     Sequential driver draft:
86     strongest available driver signs first, then the next, and so on.
87     """
88     teams_list = list(marketing_dict.keys())
89     seats_left = {t: seats_per_team for t in teams_list}
90     roster = {t: [] for t in teams_list}
91
92     drivers_sorted = sorted(drivers_list, key=lambda x: x[1], reverse=True)
93
94     for driver_name, ability in drivers_sorted:
95         eligible = [t for t in teams_list if seats_left[t] > 0]
96         if not eligible:
97             break
98
99         probs = contest_probs(marketing_dict, eligible, h_param)
100         chosen_team = rng.choice(eligible, p=probs)
101
102         roster[chosen_team].append((driver_name, ability))
103         seats_left[chosen_team] -= 1

```

```

103
104     return roster
105
106
107 # =====
108 # Performance and reliability model
109 # =====
110
111 def circuit_coefficients(circuit_type):
112     if circuit_type == "balanced":
113         return 1.0, 1.0, 1.0
114     if circuit_type == "driver":
115         return 2.0, 1.0, 1.0
116     if circuit_type == "chassis":
117         return 1.0, 2.0, 1.0
118     if circuit_type == "engine":
119         return 1.0, 1.0, 2.0
120     if circuit_type == "chassis_engine":
121         return 1.0, 2.0, 2.0
122     if circuit_type == "driver_chassis":
123         return 2.0, 2.0, 1.0
124     return 1.0, 1.0, 1.0
125
126
127 def performance_score(driving, chassis, engine, circuit_type):
128     """
129     Brief-based performance law with Setup fixed at zero.
130     """
131     c_d, c_c, c_e = circuit_coefficients(circuit_type)
132     numerator = c_d * driving + c_c * chassis + c_e * engine
133     denominator = c_d + c_c + c_e
134     return 3.0 * numerator / denominator
135
136
137 def p_dnf_from_reliability(r):
138     """
139     Mechanical-failure law from the competition brief.
140     """
141     val = 1.0 - math.erf(0.7 * r - 1.8)
142     p = (val * val) / 5.0
143     return max(0.0, min(1.0, p))
144
145
146 # =====
147 # Race and season simulation
148 # =====
149
150 def run_one_race(roster, teams_dict, circuit, rng):
151     """
152     Simulate one race:
153     1) compute performance scores,
154     2) rank entries,
155     3) apply DNFs,
156     4) award points to finishers.
157     """
158     entries = []
159

```

```

160     for team_name, signed_drivers in roster.items():
161         chassis = teams_dict[team_name]["chassis"]
162         engine = teams_dict[team_name]["engine"]
163         reliability = teams_dict[team_name]["reliability"]
164
165         for driver_name, ability in signed_drivers:
166             score = performance_score(
167                 driving=ability,
168                 chassis=chassis,
169                 engine=engine,
170                 circuit_type=circuit["type"],
171             )
172             entries.append({
173                 "team": team_name,
174                 "score": score,
175                 "reliability": reliability,
176             })
177
178     entries.sort(key=lambda x: x["score"], reverse=True)
179
180     finishers = []
181     for entry in entries:
182         if rng.random() >= p_dnf_from_reliability(entry["reliability"]):
183             finishers.append(entry)
184
185     race_points = {t: 0 for t in teams_dict}
186     for pos, entry in enumerate(finishers):
187         pts = POINTS_16[pos] if pos < len(POINTS_16) else 0
188         race_points[entry["team"]] += pts
189
190     return race_points
191
192
193 def make_teams(teamA_alloc, opp_alloc=None):
194     """
195     Build the eight-team championship field.
196     Team A uses teamA_alloc; all opponents use opp_alloc.
197     """
198     if opp_alloc is None:
199         opp_alloc = OPP_BASE
200
201     def check_budget(alloc):
202         total = (
203             alloc["marketing"]
204             + alloc["chassis"]
205             + alloc["engine"]
206             + alloc["reliability"]
207         )
208         if abs(total - TOTAL_BUDGET) > 1e-9:
209             raise ValueError(f"Allocation must sum to {TOTAL_BUDGET}. Got {total}")
210
211     check_budget(teamA_alloc)
212     check_budget(opp_alloc)
213
214     return {
215         "Team A (my team)": teamA_alloc.copy(),
216         "Team B": opp_alloc.copy(),

```

```

217     "Team C": opp_alloc.copy(),
218     "Team D": opp_alloc.copy(),
219     "Team E": opp_alloc.copy(),
220     "Team F": opp_alloc.copy(),
221     "Team G": opp_alloc.copy(),
222     "Team H": opp_alloc.copy(),
223 }
224
225
226 def run_one_season(teams_dict, rng):
227     """
228     Simulate one season and return:
229     - Team A season points
230     - whether Team A wins the constructors' championship
231     """
232     marketing_dict = {t: teams_dict[t]["marketing"] for t in teams_dict}
233     roster = sign_drivers_sequential(marketing_dict, drivers, SEATS_PER_TEAM, h, rng
234                                     )
235
236     team_points = {t: 0 for t in teams_dict}
237     for circuit in circuits:
238         race_pts = run_one_race(roster, teams_dict, circuit, rng)
239         for t in team_points:
240             team_points[t] += race_pts[t]
241
242     winner = max(team_points.items(), key=lambda x: x[1])[0]
243     return team_points["Team A (my team)"], (winner == "Team A (my team)")

```

B Objective evaluation and search code

This appendix contains selected excerpts of the code used to evaluate the two main objectives, J_{mean} and J_{win} , and to perform the optimisation search discussed in Sections 3.3 and 3.4. It includes the routines for estimating mean season points and win probability by simulation, together with the two-stage search procedure consisting of coarse screening and targeted local refinement.

The following excerpt shows the objective-evaluation routines together with the coarse-grid search logic and the local-refinement code used around the win-focused region.

```
1 import numpy as np
2
3 from appendix_A_core_model import TOTAL_BUDGET, make_teams, run_one_season
4
5 # =====
6 # Appendix B: objective evaluation and search
7 # =====
8
9
10 def estimate_mean_points(teamA_alloc, n_seasons, seed=123):
11     """
12     Estimate expected Team A season points with a 95% confidence interval.
13     """
14     rng = np.random.default_rng(seed)
15     teams = make_teams(teamA_alloc)
16
17     pts = np.zeros(n_seasons, dtype=float)
18     for k in range(n_seasons):
19         pts[k], _ = run_one_season(teams, rng)
20
21     mean = pts.mean()
22     sd = pts.std(ddof=1)
23     se = sd / np.sqrt(n_seasons)
24
25     return {
26         "mean": mean,
27         "se": se,
28         "ci_low": mean - 1.96 * se,
29         "ci_high": mean + 1.96 * se,
30     }
31
32
33 def estimate_winprob(teamA_alloc, n_seasons, seed=123):
34     """
35     Estimate Team A constructors' win probability with a 95% confidence interval.
36     """
37     rng = np.random.default_rng(seed)
38     teams = make_teams(teamA_alloc)
39
40     wins = 0
41     for _ in range(n_seasons):
42         _, win = run_one_season(teams, rng)
43         wins += 1 if win else 0
44
45     p = wins / n_seasons
46     se = np.sqrt(p * (1.0 - p) / n_seasons)
47
```

```

48     return {
49         "p": p,
50         "se": se,
51         "ci_low": p - 1.96 * se,
52         "ci_high": p + 1.96 * se,
53     }
54
55
56 def evaluate_allocation(teamA_alloc, n_seasons, seed=123):
57     """
58     Evaluate one allocation under the two main objectives:
59     - J_mean: expected season points
60     - J_win: probability of winning the constructors' championship
61     """
62     rng = np.random.default_rng(seed)
63     teams = make_teams(teamA_alloc)
64
65     pts = np.zeros(n_seasons, dtype=float)
66     wins = 0
67
68     for k in range(n_seasons):
69         pts[k], win = run_one_season(teams, rng)
70         wins += 1 if win else 0
71
72     mean_points = pts.mean()
73     sd_points = pts.std(ddof=1)
74     p_win = wins / n_seasons
75     q10 = np.quantile(pts, 0.10)
76     q90 = np.quantile(pts, 0.90)
77
78     return {
79         "mean_points": mean_points,
80         "sd_points": sd_points,
81         "p_win_constructors": p_win,
82         "q10": q10,
83         "q90": q90,
84     }
85
86
87 def generate_allocations(step, total_budget=TOTAL_BUDGET):
88     """
89     Enumerate all allocations (m, c, e, r) on a step grid
90     such that m + c + e + r = total_budget.
91     """
92     units_total = int(round(total_budget / step))
93     allocations = []
94
95     for m_u in range(units_total + 1):
96         for r_u in range(units_total - m_u + 1):
97             for c_u in range(units_total - m_u - r_u + 1):
98                 e_u = units_total - m_u - r_u - c_u
99
100                 allocations.append({
101                     "marketing": m_u * step,
102                     "chassis": c_u * step,
103                     "engine": e_u * step,
104                     "reliability": r_u * step,

```

```

105         })
106
107     return allocations
108
109
110 def paired_winprob_diff(baseline_alloc, test_alloc, n_seasons, seed=123):
111     """
112     Paired comparison of two allocations on the binary win indicator,
113     using common random numbers.
114     Returns estimated difference in win probability:
115         P(win | baseline) - P(win | test).
116     """
117     base_rng = np.random.default_rng(seed)
118     season_seeds = base_rng.integers(0, 2**31 - 1, size=n_seasons)
119
120     diffs = np.zeros(n_seasons, dtype=float)
121
122     for k in range(n_seasons):
123         s = int(season_seeds[k])
124
125         rng1 = np.random.default_rng(s)
126         _, win1 = run_one_season(make_teams(baseline_alloc), rng1)
127
128         rng2 = np.random.default_rng(s)
129         _, win2 = run_one_season(make_teams(test_alloc), rng2)
130
131         diffs[k] = (1.0 if win1 else 0.0) - (1.0 if win2 else 0.0)
132
133     mean = diffs.mean()
134     sd = diffs.std(ddof=1)
135     se = sd / np.sqrt(n_seasons)
136
137     return {
138         "mean": mean,
139         "se": se,
140         "ci_low": mean - 1.96 * se,
141         "ci_high": mean + 1.96 * se,
142     }
143
144
145 def two_stage_search(
146     step_screen=0.5,
147     n_screen=120,
148     top_k_screen=30,
149     n_refine=8000,
150     seed=123,
151 ):
152     """
153     Two-stage search used in the project:
154     1) coarse screening over all allocations,
155     2) accurate re-evaluation of shortlisted candidates.
156     """
157     candidates = generate_allocations(step_screen)
158
159     screen_results = []
160     for i, alloc in enumerate(candidates):
161         metrics = evaluate_allocation(alloc, n_screen, seed=seed + i)

```

```

162     screen_results.append(**alloc, **metrics})
163
164     top_mean = sorted(
165         screen_results, key=lambda row: row["mean_points"], reverse=True
166     )[:top_k_screen]
167     top_win = sorted(
168         screen_results, key=lambda row: row["p_win_constructors"], reverse=True
169     )[:top_k_screen]
170
171     shortlisted = []
172     seen = set()
173     for row in top_mean + top_win:
174         key = (
175             row["marketing"],
176             row["chassis"],
177             row["engine"],
178             row["reliability"],
179         )
180         if key not in seen:
181             shortlisted.append({
182                 "marketing": row["marketing"],
183                 "chassis": row["chassis"],
184                 "engine": row["engine"],
185                 "reliability": row["reliability"],
186             })
187             seen.add(key)
188
189     refined_results = []
190     for i, alloc in enumerate(shortlisted):
191         metrics = evaluate_allocation(alloc, n_refine, seed=10_000 + seed + i)
192         refined_results.append(**alloc, **metrics})
193
194     return screen_results, refined_results
195
196
197 def local_refinement_rows(
198     baseline_alloc,
199     m_values,
200     fixed_r=2.5,
201     fixed_c=0.0,
202     n_mean=5000,
203     n_paired=3000,
204     seed=123,
205 ):
206     """
207     Local refinement around a win-focused allocation.
208     Marketing is varied while reliability and chassis are fixed;
209     engine is adjusted to preserve the total budget.
210     """
211     rows = []
212
213     for m in m_values:
214         test_alloc = {
215             "marketing": float(m),
216             "chassis": float(fixed_c),
217             "reliability": float(fixed_r),
218             "engine": float(TOTAL_BUDGET - m - fixed_r - fixed_c),

```

```

219     }
220
221     pts_stats = estimate_mean_points(test_alloc, n_mean, seed)
222     win_stats = estimate_winprob(test_alloc, n_mean, seed)
223     win_diff = paired_winprob_diff(baseline_alloc, test_alloc, n_paired, seed)
224
225     rows.append({
226         "marketing": test_alloc["marketing"],
227         "chassis": test_alloc["chassis"],
228         "engine": test_alloc["engine"],
229         "reliability": test_alloc["reliability"],
230         "mean_points": pts_stats["mean"],
231         "mean_ci_low": pts_stats["ci_low"],
232         "mean_ci_high": pts_stats["ci_high"],
233         "p_win": win_stats["p"],
234         "win_ci_low": win_stats["ci_low"],
235         "win_ci_high": win_stats["ci_high"],
236         "paired_diff_mean": win_diff["mean"],
237         "paired_diff_ci_low": win_diff["ci_low"],
238         "paired_diff_ci_high": win_diff["ci_high"],
239     })
240
241     return rows

```

C Robustness and opponent-sensitivity code

This appendix contains selected excerpts of the code used for the robustness tests and opponent-sensitivity analysis in Section 4. In particular, it documents the one-at-a-time perturbation rule, the paired comparison routines, and the opponent-style simulation logic. The final opponent-style setup includes the balanced profile together with marketing-heavy, chassis-heavy, engine-heavy and reliability-heavy stress-test profiles.

The following excerpt shows the perturbation logic and the opponent-sensitivity routines used for the final figures.

```
1 import numpy as np
2
3 from appendix_A_core_model import TOTAL_BUDGET, make_teams, p_dnf_from_reliability,
   run_one_season
4 from appendix_B_search import estimate_mean_points
5
6 # =====
7 # Appendix C: robustness and opponent sensitivity
8 # =====
9
10
11 def perturb_one_at_a_time(base_alloc, key, frac_change):
12     """
13     Perturb one component by a given percentage and preserve the total budget
14     by rebalancing chassis and engine.
15
16     This is the rule used in the robustness analysis.
17     """
18     alloc = base_alloc.copy()
19
20     old = alloc[key]
21     new = max(0.0, old * (1.0 + frac_change))
22     alloc[key] = new
23
24     delta = new - old
25
26     if delta >= 0:
27         c_try = alloc["chassis"] - delta / 2.0
28         e_try = alloc["engine"] - delta / 2.0
29
30         if c_try >= 0 and e_try >= 0:
31             alloc["chassis"] = c_try
32             alloc["engine"] = e_try
33         else:
34             e_only = alloc["engine"] - delta
35             if e_only < 0:
36                 return None
37             alloc["engine"] = e_only
38     else:
39         alloc["chassis"] += (-delta) / 2.0
40         alloc["engine"] += (-delta) / 2.0
41
42     total = (
43         alloc["marketing"]
44         + alloc["chassis"]
45         + alloc["engine"]
```

```

46         + alloc["reliability"]
47     )
48     correction = (TOTAL_BUDGET - total) / 2.0
49     alloc["chassis"] += correction
50     alloc["engine"] += correction
51
52     if alloc["chassis"] < -1e-9 or alloc["engine"] < -1e-9:
53         return None
54
55     alloc["chassis"] = max(0.0, alloc["chassis"])
56     alloc["engine"] = max(0.0, alloc["engine"])
57
58     return alloc
59
60
61 def paired_points_diff(baseline_alloc, test_alloc, n_seasons, seed=123):
62     """
63     Paired comparison of two allocations on Team A season points.
64     Returns baseline - test.
65     """
66     base_rng = np.random.default_rng(seed)
67     season_seeds = base_rng.integers(0, 2**31 - 1, size=n_seasons)
68
69     diffs = np.zeros(n_seasons, dtype=float)
70
71     for k in range(n_seasons):
72         s = int(season_seeds[k])
73
74         rng1 = np.random.default_rng(s)
75         pts1, _ = run_one_season(make_teams(baseline_alloc), rng1)
76
77         rng2 = np.random.default_rng(s)
78         pts2, _ = run_one_season(make_teams(test_alloc), rng2)
79
80         diffs[k] = pts1 - pts2
81
82     mean = diffs.mean()
83     sd = diffs.std(ddof=1)
84     se = sd / np.sqrt(n_seasons)
85
86     return {
87         "mean": mean,
88         "se": se,
89         "ci_low": mean - 1.96 * se,
90         "ci_high": mean + 1.96 * se,
91     }
92
93
94 def paired_winprob_diff(baseline_alloc, test_alloc, n_seasons, seed=123):
95     """
96     Paired comparison of two allocations on the binary constructors-win indicator.
97     Returns baseline - test.
98     """
99     base_rng = np.random.default_rng(seed)
100     season_seeds = base_rng.integers(0, 2**31 - 1, size=n_seasons)
101
102     diffs = np.zeros(n_seasons, dtype=float)

```

```

103
104     for k in range(n_seasons):
105         s = int(season_seeds[k])
106
107         rng1 = np.random.default_rng(s)
108         _, win1 = run_one_season(make_teams(baseline_alloc), rng1)
109
110         rng2 = np.random.default_rng(s)
111         _, win2 = run_one_season(make_teams(test_alloc), rng2)
112
113         diffs[k] = (1.0 if win1 else 0.0) - (1.0 if win2 else 0.0)
114
115     mean = diffs.mean()
116     sd = diffs.std(ddof=1)
117     se = sd / np.sqrt(n_seasons)
118
119     return {
120         "mean": mean,
121         "se": se,
122         "ci_low": mean - 1.96 * se,
123         "ci_high": mean + 1.96 * se,
124     }
125
126
127 def robustness_scan(
128     baseline_alloc,
129     keys=("marketing", "reliability"),
130     fracs=(-0.10, -0.05, 0.05, 0.10),
131     n_mean=5000,
132     n_paired=5000,
133     seed=123,
134 ):
135     """
136     One-at-a-time robustness analysis around a baseline allocation.
137     """
138     rows = []
139
140     for key in keys:
141         for frac in fracs:
142             test_alloc = perturb_one_at_a_time(baseline_alloc, key, frac)
143
144             if test_alloc is None:
145                 rows.append({
146                     "key": key,
147                     "frac": frac,
148                     "feasible": False,
149                 })
150                 continue
151
152             stats = estimate_mean_points(test_alloc, n_mean, seed)
153             pts_diff = paired_points_diff(baseline_alloc, test_alloc, n_paired, seed)
154
155             rows.append({
156                 "key": key,
157                 "frac": frac,
158                 "feasible": True,
159                 "marketing": test_alloc["marketing"],

```

```

160         "chassis": test_alloc["chassis"],
161         "engine": test_alloc["engine"],
162         "reliability": test_alloc["reliability"],
163         "p_dnf": p_dnf_from_reliability(test_alloc["reliability"]),
164         "mean_points": stats["mean"],
165         "mean_ci_low": stats["ci_low"],
166         "mean_ci_high": stats["ci_high"],
167         "paired_points_diff": pts_diff["mean"],
168         "paired_points_ci_low": pts_diff["ci_low"],
169         "paired_points_ci_high": pts_diff["ci_high"],
170     })
171
172     return rows
173
174
175     opponent_styles = {
176         "Balanced (2.0,2.0,2.0,2.0)": {
177             "marketing": 2.0,
178             "chassis": 2.0,
179             "engine": 2.0,
180             "reliability": 2.0,
181         },
182         "Marketing-heavy (3.2,1.6,1.6,1.6)": {
183             "marketing": 3.2,
184             "chassis": 1.6,
185             "engine": 1.6,
186             "reliability": 1.6,
187         },
188         "Chassis-heavy (1.6,3.2,1.6,1.6)": {
189             "marketing": 1.6,
190             "chassis": 3.2,
191             "engine": 1.6,
192             "reliability": 1.6,
193         },
194         "Engine-heavy (1.6,1.6,3.2,1.6)": {
195             "marketing": 1.6,
196             "chassis": 1.6,
197             "engine": 3.2,
198             "reliability": 1.6,
199         },
200         "Reliability-heavy (1.6,1.6,1.6,3.2)": {
201             "marketing": 1.6,
202             "chassis": 1.6,
203             "engine": 1.6,
204             "reliability": 3.2,
205         },
206     }
207
208     def simulate_many(teamA_alloc, opp_alloc, n_seasons, seed=123):
209         """
210         Evaluate one Team A strategy against one opponent style.
211         """
212         rng = np.random.default_rng(seed)
213         teams = make_teams(teamA_alloc, opp_alloc=opp_alloc)
214
215         pts = np.zeros(n_seasons, dtype=float)
216         wins = 0

```

```

217
218     for k in range(n_seasons):
219         pts[k], win = run_one_season(teams, rng)
220         wins += 1 if win else 0
221
222     mean = pts.mean()
223     sd = pts.std(ddof=1)
224     se = sd / np.sqrt(n_seasons)
225     p = wins / n_seasons
226     se_p = np.sqrt(p * (1.0 - p) / n_seasons)
227
228     return {
229         "mean_points": mean,
230         "ci_points_low": mean - 1.96 * se,
231         "ci_points_high": mean + 1.96 * se,
232         "p_win": p,
233         "ci_pwin_low": p - 1.96 * se_p,
234         "ci_pwin_high": p + 1.96 * se_p,
235     }
236
237
238 def opponent_sensitivity(teamA_strategies, styles=None, n_seasons=10000, seed=123):
239     """
240     Compare multiple Team A strategies across a set of representative opponent
241     styles.
242     The default styles are one balanced profile together with marketing-heavy,
243     chassis-heavy, engine-heavy and reliability-heavy profiles.
244     """
245     if styles is None:
246         styles = opponent_styles
247
248     rows = []
249
250     for strategy_name, teamA_alloc in teamA_strategies.items():
251         for style_name, opp_alloc in styles.items():
252             out = simulate_many(teamA_alloc, opp_alloc, n_seasons, seed)
253             rows.append({
254                 "teamA_strategy": strategy_name,
255                 "opponent_style": style_name,
256                 **out,
257             })
258
259     return rows

```

Declarations of Academic Integrity & GenAI use

I am aware that the University defines plagiarism as presenting someone else's work, in whole or in part, as your own. Work means any intellectual output, and typically includes text, data, images, sound or performance.

I promise that in the attached submission I have not presented anyone else's work, in whole or in part, as my own and I have not colluded with others in the preparation of this work. Where I have taken advantage of the work of others, I have given full acknowledgement. I have not resubmitted my own work or part thereof without specific written permission to do so from the University staff concerned when any of this work has been or is being submitted for marks or credits even if in a different module or for a different qualification or completed prior to entry to the University. I have read and understood the University's published rules on plagiarism and also any more detailed rules specified at School or module level. I know that if I commit plagiarism, I can be expelled from the University and that it is my responsibility to be aware of the University's regulations on plagiarism and their importance.

I re-confirm my consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to monitor breaches of regulations, to verify whether my work contains plagiarised material, and for quality assurance purposes.

I confirm that I have declared all mitigating circumstances that may be relevant to the assessment of this piece of work and that I wish to have taken into account. I am aware of the University's policy on mitigation and the School's procedures for the submission of statements and evidence of mitigation. I am aware of the penalties imposed for the late submission of coursework.

GenAI use

I acknowledge that I have used Microsoft Copilot (<https://copilot.microsoft.com>) to help debug Python and LaTeX code for this report.

Signed: Kiran Patel

Date: 27/03/2026